



Original Software Publication

SDPTool : A tool for creating datasets and software defect predictions

Jalaj Pachouly, Swati Ahirrao, Ketan Kotecha*

Symbiosis Institute of Technology, Symbiosis International (Deemed University), Pune 412115, India



ARTICLE INFO

Article history:

Received 30 December 2021

Received in revised form 20 February 2022

Accepted 24 February 2022

Keywords:

Java Swing

REST API

GitHub

SQLite

Relational database

Open JDK

ABSTRACT

One of the most difficult and time-consuming aspects of machine learning is gathering high-quality data that can be used to train the algorithm. Historical data is a gold mine for predicting the future with a high level of confidence and accuracy in a certain field. Data is typically available in its raw form, which is in most circumstances unsuitable for machine learning applications. The work presented here introduces a tool that will considerably assist in the generation of a high-quality dataset, starting with raw data collection and progressing on to data pre-processing and validation, and finally to prediction using the selected machine learning algorithm. So far, we have built capability in the SDPTool to collect data dynamically from a GitHub repository. Data pre-processing, various defect predictions, and other machine learning settings are all in the pipeline. Once the data is ready to use, SDPTool will assist in selecting the appropriate machine learning algorithm and performs the desired prediction in the associated field. The entire process will be orchestrated in a user-friendly Java Swing application, from data collection to prediction.

© 2022 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

Code metadata

Current code version	v1.0
Permanent link to code/repository used for this code version	https://github.com/ElsevierSoftwareX/SOFTX-D-22-00008
Legal Code License	GNU General Public License
Code versioning system used	GitHub
Software code languages, tools, and services used	Java Swing
Compilation requirements, operating environments & dependencies	Open JDK, Maven, SQLite
Developer Documentation/Manual	https://github.com/jalajpachouly/SDPTool/wiki
Support email for questions	jalaj.pachouly.phd2019@sitpune.edu.in
Video demonstration of SDPTool and sample dataset created using SDPTool	https://github.com/jalajpachouly/SDPTool/

1. Motivation and significance

Early researcher on software defect prediction used a binary classification to indicate whether a software package contains bugs. Existing datasets that are publicly available were built with a binary classification mindset. We need a better dataset with additional parameters such as the severity of the defect, the time it may take to fix it, the complexities of the defect, expected resource allocation, and potential code changes to fix that defect, that will help in predicting more relevant information for software defects. The most difficult and crucial stage is to create a dataset with the required feature set and prepare it for machine

learning modeling through a series of pre-processing and data validation steps. A high-quality defect dataset that can be used for a variety of research activities is a common demand in the field of software defect prediction.

GitHub [1] is a publicly accessible platform for storing code and bugs related to it. It does contain very useful discussion around the logged defects between the developer and the end-user. Communication between the developer and the end-user about code changes can be a great source of information. It can be used to predict a variety of relevant information for planning in order to improve planning efficiency. Our work describes an approach for retrieving data from a GitHub repository and data wrangling to get the information that is important for forecasting various defect-related characteristics using a user-interface tool. A Representational State Transfer (REST) API is available on GitHub for retrieving raw data from the repository, which

* Corresponding author.

E-mail address: head@scaai.siu.edu.in (Ketan Kotecha).

is subsequently processed and stored in a relational database for analysis. The user can configure the code repository and the relational database that will hold the raw and processed data using the user interface.

NASA defect datasets is used in many studies. Shepperd et al. [2] investigated five research and emphasized the value of data quality and pre-processing for machine learning. PROMISE repository [3] contains datasets for software defect prediction capable of binary classification. Yatish et al. [4] created a public dataset using the Jira issue tracking system. Existing datasets are designed for binary classification, such as whether a code change is buggy or not. According to Pachouly et al. [5], there is a lot of interest in the research community on Software Defect Prediction. Pachouly et al. [6] also mentioned that collecting the defect prediction dataset for multi-label prediction can provide the software development team with more useful prediction information. We believe we are the first to attempt to create a generic tool for producing a multi-label dataset with a rich feature set from any GitHub repository via a user interface tool, with any number of records for software defect prediction. SDPTool will eliminate the need for researchers to experiment with pre-existing static datasets, allowing them to shift from one dataset to the next in a matter of minutes without putting in the time and effort required to develop the dataset.

2. Software description

The high-level software architecture is covered in this section. Fig. 1 depicts the high-level building blocks used in the development of SDPTool. The important relational database schema for storing the collected data in the local database is shown in Fig. 2. This schema may be adjusted in the future to fit the SDPTool's additional features and functionality, as we intend to improve it. The high-level procedures for extracting and processing data from the GitHub repository are shown in Fig. 3.

2.1. Software architecture

The components are listed as follows:

1. The Java client sends a REST API request to the GitHub server with an authentication token via HTTPS, and the GitHub server responds with a JavaScript Object Notation (JSON) response including different details about the defect as well as the necessary code changes in raw text format. The GitHub server authenticates the user and returns the JSON response for the specific project from the GitHub repository based on the request parameters supplied in the REST API call.
2. When the Java client receives a response from GitHub, it goes through the data processing stages, which include data wrangling and extracting relevant information from it to create the multi-label dataset.
3. The processed data is subsequently saved in a local database repository, where it will be utilized to train the multi-label prediction model later.

To make this process go as smoothly as possible, a well-defined user interface is created using a Java Swing application that allows users to configure the local database and set up the user authentication token, as well as set parameters to control the number of records returned from the GitHub repository.

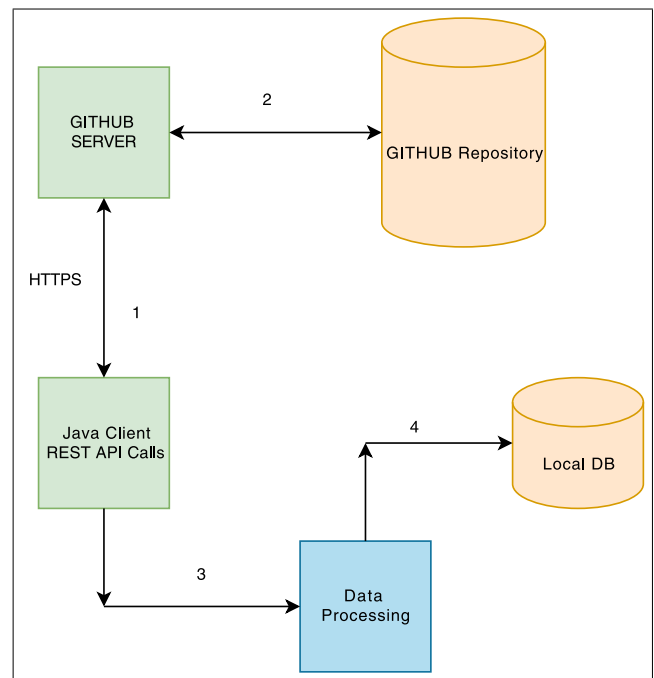


Fig. 1. Data fetching mechanism from public GitHub repository.

2.2. Software functionalities

The software's most important features are listed below:

1. Configuration: This is the initial tab that appears in the SDPTool after it has been launched, and it allows the user to manage the local database configuration as well as the remote repository from which the raw data for the prediction must be collected.
2. Database Pre-Processing and validation: This is the second tab, which assists in performing different pre-processing steps on the obtained raw dataset in order to ensure that the final data is of high quality and ready to be used in the machine learning model.
3. Feature Selection and Sample Selection: This is the third tab of the SDPTool, where the user can define the ratio of samples for training and testing the machine learning model, as well as configure features from the dataset.
4. Artificial Intelligence Techniques: This tab allows the user to select from a number of built-in Artificial Intelligence algorithms that can be used to create a prediction model.
5. Predictions: This is the last tab of the SDPTool, and it displays the expected result depending on the setting and other choices made on the preceding tabs. It will also display quality indicators for the created model, such as precision and accuracy.

2.3. Sample code snippets analysis

Rather than using REST API calls to consume the data directly, we chose to use the Object Oriented version of the GitHub API, which essentially provides domain models that define the data model for the data contained in the GitHub. The GHIssue Java class can be used to download an issue in the updated code, which could be a bug or a feature request. At a higher level, the GitHub class can be used to set up the connection to the repository and get the reference to various data objects from the GitHub repository. GitHub, GHIssue, GHRepository, GitHubBuilder, and

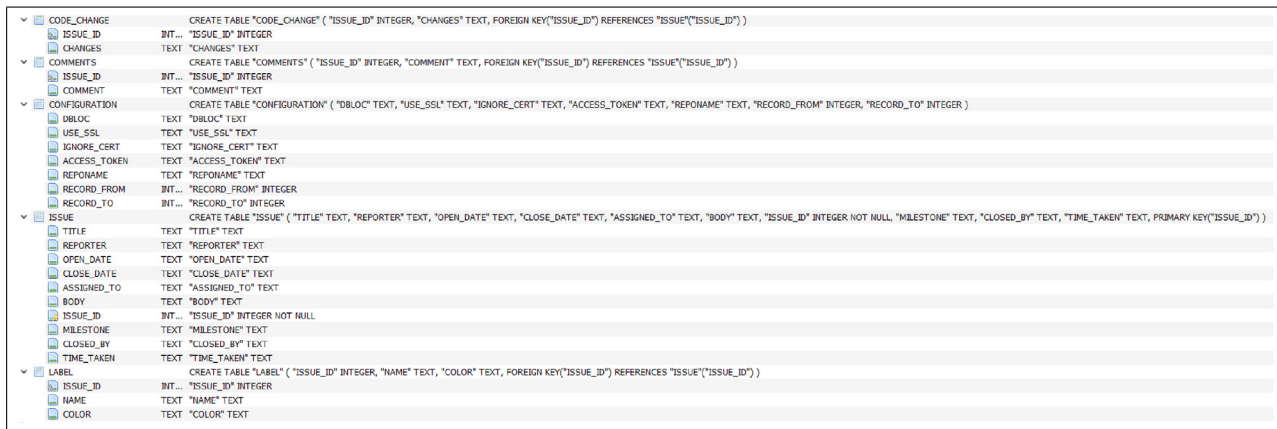


Fig. 2. Relational Database Schema for storing and processing data from GitHub repositories.

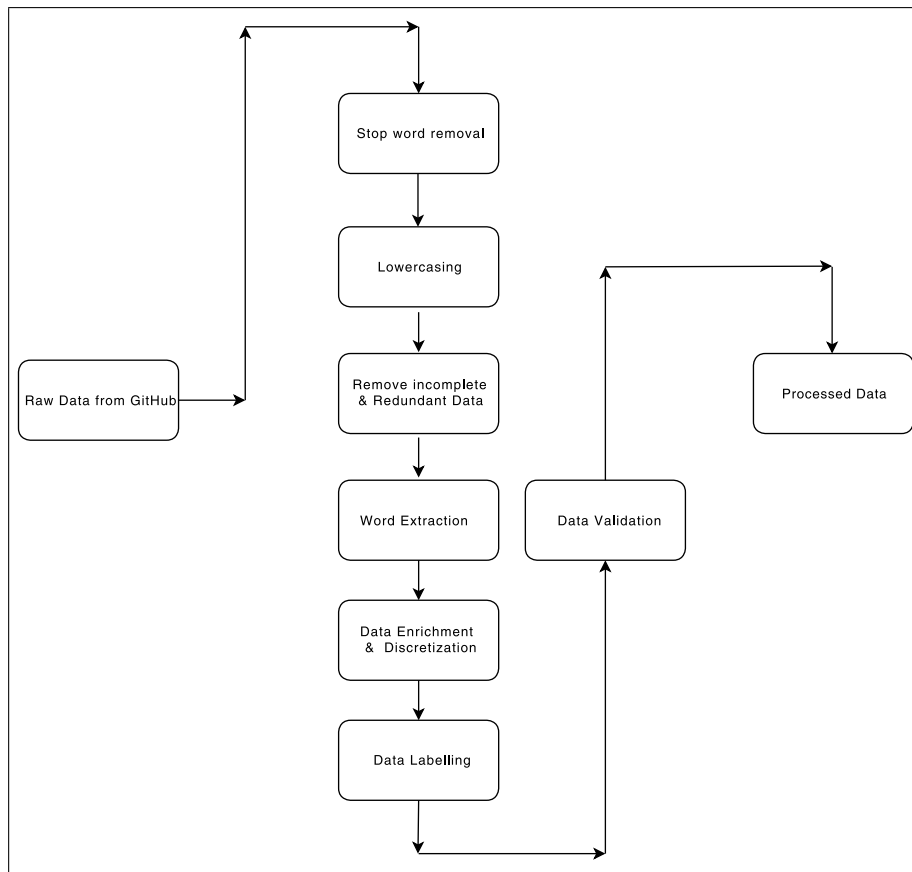


Fig. 3. Data extraction from a public GitHub repository.

GHLabel are a few more important classes that are used to fetch defect data. GHIssue is the main class that represents the defect in the GitHub repository, and it has a number of useful methods for retrieving important information about the defect, such as its status, assignee, and labels, as well as the code that was updated to correct it. They can be thought of as wrapper classes that mask REST API calls from high-level code. To connect to a GitHub repository, we must first generate a personal access token for a specific user with certain rights from the GitHub site, so that the user can retrieve the relevant details using the generated access token. The data from GitHub is kept in the relational database SQLite [7], which is a lightweight database that is ideal

for storing the defect prediction dataset as well as pre-processing and preparing for various software defect predictions.

3. Illustrative examples

Before utilizing SDPTool to create a custom Defect Prediction dataset, we need to make sure the system has the necessary software setup. Required software are:

1. Java (Version 8 or higher)
2. SQLite (Version: 3.12.2)
3. Maven (Version 3 or higher) (For Building the code)

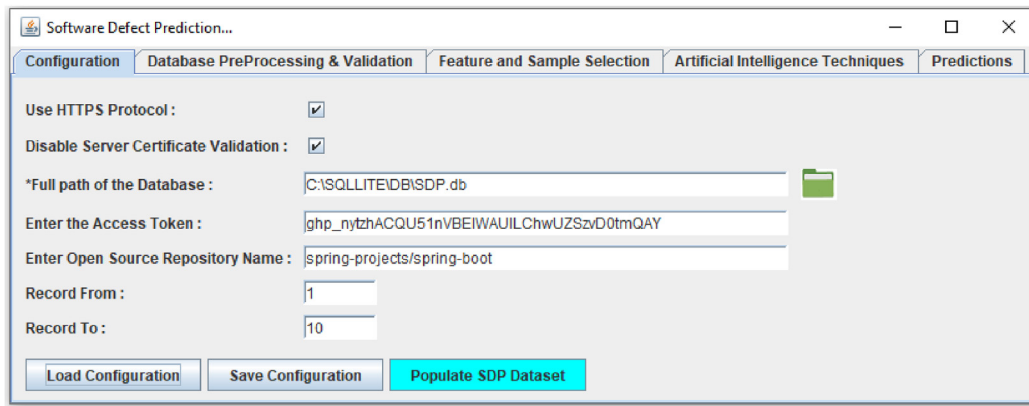


Fig. 4. User interface with sample configuration parameters to create the prediction dataset using public GitHub repository.

Users can follow step-by-step video instructions [8] to set up and run the tool, as well as generate custom datasets on demand from a remote GitHub repository. The steps in a logical order are listed below.

1. Download the source code
2. Build the tool (Refer wiki page [9])
3. Launch the tool
4. Provide Configuration
5. Create Custom Datasets
6. Run data pre-processing and validation

Instructions for building and launching the tool can be found on the Wiki page [9].

3.1. Configuring the tool

We must configure the SDPTool to refer to a local SQLite database to store the data and a remote GitHub repository from which we wish to fetch the defect prediction dataset before we can use it. In order to connect to the GitHub repository, we must also provide the access token. We must also specify how many records from the remote repository we want to retrieve. The various configuration parameters required before collecting data from the remote repository are shown in Fig. 4. The user can retrieve as many records as he wants, and he can choose whether or not to use the secure protocol for communication, as well as whether or not to validate the GitHub server's digital certificate before making a secure connection with the repository.

3.2. Populating dataset

After we are done configuring the SDPTool, we can start populating the dataset, as shown in Figs. 5 and 6. To populate the dataset, click the Populate SDP Dataset button, which will turn red and display the status of the dataset populating operation on the progress bar.

3.3. Dataset pre-processing and validation

After the data has been populated, we can move to the data pre-processing and validation tab and complete the dataset pre-processing and validation described in Figs. 7 and 8.

3.4. Sample dataset generated using SDPTool

Sample datasets [10] generated with SDPTool are available online. The sample datasets generated with SDPTool are summarized in Table 1.

4. Impact

SDPTool will make the application of machine learning to numerous new fields convenient and straightforward because it allows researchers to rapidly set up a variety of datasets and prepare them for usage in machine learning models. Changing the data collection options from the SDPTool makes it simple to run multiple experiments, resulting in a variety of outcomes that would otherwise require a lot of time and effort to prepare manually. With a user-interface SDPTool, the user may focus on the functional requirements at hand instead of boilerplate code, which is handled in the background by the SDPTool. Existing studies that want to extend their findings can use SDPTool to run several iterations of their algorithm on different datasets of varying sizes, as creating a new dataset is an easy process using SDPTool. Various additional analyses can be undertaken in the existing research by focusing on different attributes such as features, samples, and dataset quality, such as selecting a separate dataset or a different machine learning algorithm with just a click of a button. Setting up the dataset, features, and sample with good quality is the most time-consuming and demanding task for any machine learning prediction. Automating this entire process is a tremendous assist in speeding up the research process. This enables researchers to conduct their studies more smoothly and obtain faster results. We can see that SDPTool can be used by any individual researcher or institution that has to accomplish a work that requires machine learning. This is how we expect SDPTool to assist them in kicking off their task with less friction and obtaining the intended outcome more rapidly. Machine learning's commercial application has grown significantly, but ordinary users are still ill-equipped to use it unless they undergo specialized training in multiple languages and modeling methodologies. Any user with a basic understanding of the user interface tool will be able to profit from machine learning prediction in their job, as the tool does not require any programming experience from the end-user.

5. Conclusions

Good quality data is the most important part of any machine learning requirement. Our platform allows researchers to gather and process data on-demand with quality checks, allowing them to perform several iterations of data collection and data pre-processing procedures to achieve high-quality data. Experiments in the field of software defect prediction have yielded positive results, demonstrating that we can collect data for multi-label software defect prediction for any GitHub repository. We would like to make SDPTool more user-friendly by allowing users to

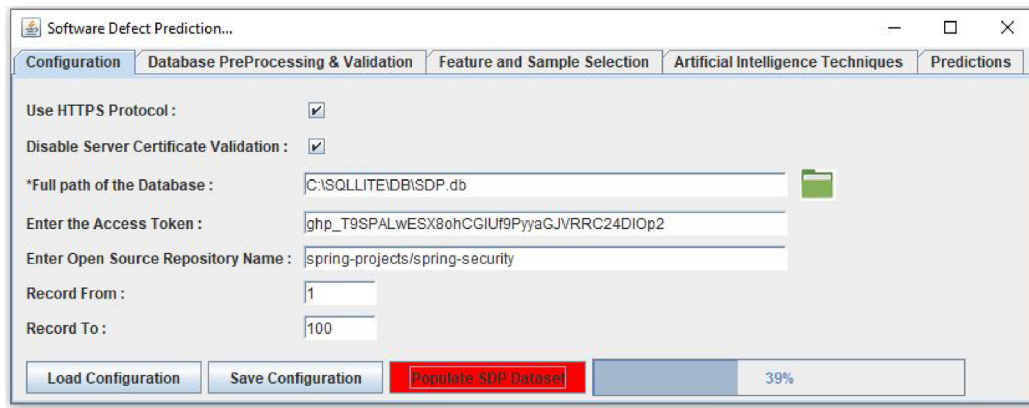


Fig. 5. Population of the dataset – In Progress.

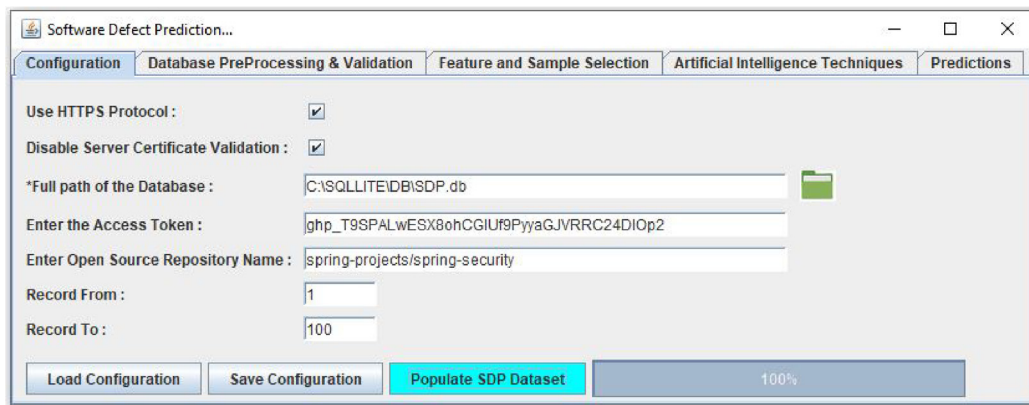


Fig. 6. Population of the dataset – Completed.

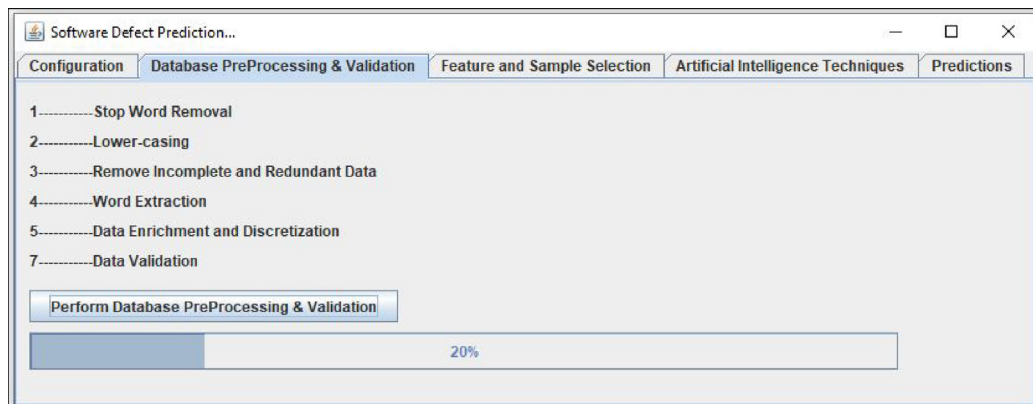


Fig. 7. Data Pre Processing – In Progress.

Table 1
Sample datasets generated using SDPTool.

Dataset (Zip Files)	Size (KB)	Modified classes	Modified packages	Comments	Authors	Rows
Spring-Batch [10]	761	62	49	117	13	100
Spring-Boot [10]	911	27	30	288	9	100
Spring-Data-Jdbc [10]	362	58	27	158	13	100
Spring-Framework [10]	4590	83	70	268	21	100
Spring-Security [10]	450	68	57	301	17	100

change data quality indicators and word extraction policies to get greater control over the processed data. We would like to improve SDPTool so that it can create end-to-end machine learning predictions from data collection through prediction.

6. Future work

We are planning to orchestrate the complete work flow for multi-label defect prediction using the proposed SDPTool. As of

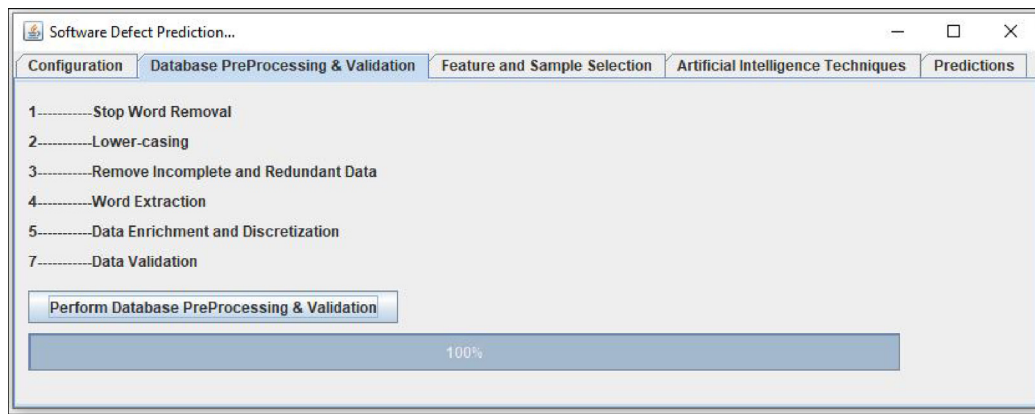


Fig. 8. Data Pre Processing — Completed.

now we have achieved dynamic data collection for any GitHub repository based on the input from the user using user interface. Here is the further plan:

1. Data pre-processing and Validation
2. Feature and Sample Selection
3. Artificial Intelligence Techniques
4. Multi-label defect predictions

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the Research Support Fund of Symbiosis International (Deemed University).

References

- [1] Github API for java. 2021, Available online: <https://github-api.kohsuke.org>. (Accessed 18 December 2021).
- [2] Shepperd M, Song Q, Sun Z, Mair C. Data quality: Some comments on the nasa software defect datasets. *IEEE Trans Softw Eng* 2013;39(9):1208–15. <http://dx.doi.org/10.1109/TSE.2013.11>.
- [3] The PROMISE software engineering repository. 2022, Available online: <http://promise.site.uottawa.ca/SERepository/datasets-page.html>. (Accessed 20 February 2022).
- [4] Yatish S, Jiarpakdee J, Thongtanunam P, Tantithamthavorn C. Mining software defects: Should we consider affected releases? In: 2019 IEEE/ACM 41st International conference on software engineering. IEEE; 2019, p. 654–65. <http://dx.doi.org/10.1109/ICSE.2019.00075>.
- [5] Pachouly J, Ahirrao S, Kotecha K. A bibliometric survey on the reliable software delivery using predictive analysis. *Libra Philos Pract* 2020; 1–27.
- [6] Pachouly J, Ahirrao S, Kotecha K, Selvachandran G, Abraham A. A systematic literature review on software defect prediction using artificial intelligence: datasets, data validation methods, approaches, and tools. *Engineering Applications of Artificial Intelligence* 2022; <http://dx.doi.org/10.1016/j.engappai.2022.104773>.
- [7] DB browser for SQLite. 2021, Available online: <https://sqlitebrowser.org>. (Accessed 18 December 2021).
- [8] Video demonstration (video-demo.zip) of setting up and using the tool. 2022, available online: <https://github.com/jalajpachouly/SDPTool/> (Accessed 20 February 2022).
- [9] Documentation for setting up and using the tool. 2022, available online: <https://github.com/jalajpachouly/SDPTool/wiki>. (Accessed 20 February 2022).
- [10] Datasets created using SDPTool (custom-dataset.zip). 2022, available online : <https://github.com/jalajpachouly/SDPTool/>. (Accessed 20 February 2022).